

# Example Walkthrough

Using Ring-AllReduce on 3 GPUs

# 1. Parsing NCCL Logs

- The NCCL logs can be enabled to setting the following env vars:
  - `NCCL_DEBUG=TRACE`
  - `NCCL_DEBUG_SUBSYS=ALL`
  - `NCCL_DEBUG_FILE=<custom file path>.%h.%p`
    - NCCL will populate %h and %p with host name and process ID
- If we're running a Distributed training scenario over 3 GPUs, there will be 3 separate NCCL log files, each recording the NCCL function calls on device.

# 1. Parsing NCCL Logs

- What AllReduce function call look like in NCCL logs:
  - NCCL INFO AllReduce: opCount 2 sendbuff 0x7f4992000000 recvbuff 0x7f4992000000 count 16777216 datatype 7 op 0 root 0 comm 0x7f4781b0cc80 [nranks=3] stream 0x55a36fe79fd0
- After Parsing:
  - NcclFunction:
    - Func name: AllReduce
    - Number of elements: 16777216
    - Data type: 7, which is ncclFloat32
    - Data size:  $16777216 * 4 = 67108864$  bytes

# 1. Parsing NCCL Logs

- Moreover, NCCL logs also identify who are the neighbors of this device in the Ring or Tree topology. The log will look something like this:
  - NCCL INFO Ring 00 : 2 -> 0 -> 1
- This means GPU 0 is receiving data from GPU 2 and sending data to GPU 1 in a Ring topology.

# 1. Parsing NCCL Logs

- After gathering logs from all GPUs, we can construct the full Ring, based on the info from each NCCL log files:
  - From GPU 0: NCCL INFO Ring 00 : 2 -> 0 -> 1
  - From GPU 1: NCCL INFO Ring 00 : 0 -> 1 -> 2
  - From GPU 2: NCCL INFO Ring 00 : 1 -> 2 -> 0
  - Hence, the full Ring topology is 0->1->2->0

## 2. Parse Torch Logs

- Similar to NCCL logs, Torch profiler will generate 3 files, one for each device
- The torch profiler can be enabled by adding the following to the code:
  - `activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]`
  - `with profile(activities=activities, on_trace_ready=torch.profiler.tensorboard_trace_handler(<a directory to store output>)) as prof:`
    - `<your code>`

## 2. Parse Torch Logs

- The torch profiler output is in JSON format, the AllReduce function call will look like this:
  - { "ph": "X", "cat": "kernel", "name": "ncclDevKernel\_AllReduce\_Sum\_f32\_RING\_LL(ncclDevComm\*, unsigned long, ncclWork\*)", "pid": 0, "tid": 25, "ts": 7650647648691.744, "dur": 555119.318}
- After Parsing:
  - TorchEvent:
    - Func name: AllReduce
    - Start time: 7650647648691.744
    - Duration: 555119.318 microseconds
    - Data type: f32 (which probably means float32)
    - Topology: Ring

### 3. Combine NCCL and Torch

- Each NCCL calls in NCCL log will have a corresponding event in Torch profiler output, so in this step, we will combine the corresponding events into a single object.
- The system will identify corresponding events based on order.
  - i.e., the first AllReduce in NCCL log will correspond to the first AllReduce in Torch profiler output.

### 3. Combine NCCL and Torch

- The final result will look something like this:
  - NcclFunction (after combining info from Torch):
    - Func name: AllReduce
    - Number of elements: 16777216
    - Data type: ncclFloat32
    - Data size: 67108864 bytes
    - Start time: 7650647648691.744
    - Duration: 555119.318 microseconds
    - Topology: Ring

## 4. Construct Collective Group across Devices

- For an AllReduce operation over 3 GPUs, there will be an AllReduce function call on each GPU. Hence, after parsing all the logs, we will have 3 NcclFunction objects referring to the same AllReduce operation.
- In this step, we will find the NcclFunction object on each device that refers to the same AllReduce operation, and combine them into a single Collective Group.
- The system will identify corresponding objects based on order:
  - i.e., the first AllReduce on GPU 0 will correspond to first AllReduce on GPU 1, which will correspond to first AllReduce on GPU 2.

## 4. Construct Collective Group across Devices

- The final result will look something like the following.
  - Collective Group:
    - Func name: AllReduce
    - Number of elements: 16777216
    - Data type: ncclFloat32
    - Data size: 67108864 bytes
    - Topology: Ring
    - GPU 0: Start time: 7650647648691.744 / Duration: 555119.318 microseconds
    - GPU 1: Start time: 7650648648691.744 / Duration: 555120.320 microseconds
    - GPU 2: Start time: 7650647848691.744 / Duration: 556000.315 microseconds
- Essentially, a Collective Group represents the actual Collective operation performed across GPUs.

## 5. Infer Data Flows

- In this step, we will infer all the data flows from the Collective Group.
- The data flow inference of each collective operation is based on my understanding after reading how NCCL implements each collective operation.
- Thus, this part will need to be updated if NCCL decides to change its implementation, or any of my current understanding is incorrect.

## 5. Infer Data Flows

- We essentially hard-coded how data flows look like under each collective operation.
- For example, for Ring-AllReduce, based on the Ring structure we discovered from NCCL logs, here's how data are transferred:
  - Starts on GPU 0: 0 -> 1 -> 2 -> 0 -> 1
  - Starts on GPU 1: 1 -> 2 -> 0 -> 1 -> 2
  - Starts on GPU 2: 2 -> 0 -> 1 -> 2 -> 0
  - First each GPU will send their data to all others via the Ring, and then each GPU will broadcast the All-Reduced results to others again via the Ring.

## 5. Infer Data Flows

- The data flow object will look something like this:
  - Data flow:
  - Src device: GPU 0
  - Dst device: GPU 1
  - Data size: 67108864 bytes
  - Dependencies: <list of data flows that this one depends on>
- At the end of this step, we will attach all the inferred data flow to the Collective Group.

## 6. Simulator Verification (Hypothetically)

- Based on my understanding of the simulator, I'm assuming that given the following info:
  - Data flow:
  - Src device: GPU 0
  - Dst device: GPU 1
  - Data size: 67108864 bytes
- The simulator should be able to estimate this data flow's duration
  - Let's say it's 10 second

## 6. Simulator Verification (Hypothetically)

- I'm also assuming that given the dependencies between the flows:
  - Let's say data flow 1->2 depends on data flow 0->1
- The simulator should also be able to estimate start time and end time of each data flow:
  - Let's say both 1->2 and 0->1 take 10 seconds, so if 0->1 starts at  $T_0$  then 1->2 should start after  $T_0 + 10s$ .

## 6. Simulator Verification (Hypothetically)

- Therefore, given the Ring-AllReduce example previously:
  - Starts on GPU 0: 0 -> 1 -> 2 -> 0 -> 1
  - Starts on GPU 1: 1 -> 2 -> 0 -> 1 -> 2
  - Starts on GPU 2: 2 -> 0 -> 1 -> 2 -> 0
- From my understanding, the AllReduce function on GPU 0 will end once it has sent and received all necessary data.
  - In other words, once the underlined data flows are done, AllReduce function on GPU 0 should end.
- Since the simulator is capable to estimate when each data flow starts and ends, it should also be capable to know when the AllReduce function on GPU0 starts and ends.

## 6. Simulator Verification (Hypothetically)

- From the previous steps, we know that the Collective Group will contain the real-world measurement of when AllReduce function on GPU0 starts and ends:
  - Collective Group:
    - GPU 0: Start time: 7650647648691.744 / Duration: 555119.318 microseconds
- We can verify the simulated start/end time of AllReduce on GPU0 against this value.

# 7. Visualization

- I'll not go through this in detail since it's irrelevant to the simulator.
- But the main takeaway is that the system is capable of generating traces from data flows that can be visualized in a website called Perfetto UI: <https://ui.perfetto.dev/>

The End.